
Deploy della policy SAC su STM32F407VET6

Appunti di ingegneria

Progetto fan-sac-controller (controllo in forza di un fan con Reinforcement Learning)
Autore Matteo Faggian
Periodo ottobre 2026
Codice stm32/rete/, stm32/firmware/FanStm32/

Come leggere questi appunti Descrivono tutto ciò che è stato fatto per portare la policy addestrata in simulazione su un microcontrollore: scelta dell'hardware, traduzione della rete in C con verifica numerica, configurazione del clock e della USB, creazione del progetto, compilazione e firmware di test. Ogni numero riportato è stato misurato o letto dai file del progetto; dove un risultato non è ancora disponibile è scritto esplicitamente.

Indice

1	Obiettivo e architettura	2
2	Scelta del microcontrollore	2
2.1	Dimensione della rete	2
2.2	Confronto tra le schede	2
2.3	Bilancio di tempo	3
3	La policy come funzione	3
3.1	Cosa calcola l'actor in modalità deterministica	3
3.2	Disposizione dei pesi in memoria	3
4	Esportazione della rete in C	3
4.1	Perché a mano e non con X-CUBE-AI	3
4.2	Script di esportazione	4
4.3	Inferenza in C	5
4.4	Verifica sul PC	6
5	Clock a 168 MHz	7
5.1	Catena del PLL	7
5.2	Bus	7
5.3	Errore incontrato: $N = 136$	7
6	USB: caricamento (DFU) e comunicazione (CDC)	7
7	Il progetto CubeIDE	8
7.1	Struttura	8
7.2	Il linker script	8
7.3	Problemi incontrati nella creazione del progetto	8
8	Compilazione	9
8.1	Dal sorgente all'eseguibile	9
8.2	Compilazione da riga di comando	9
8.3	Occupazione di memoria	10
9	Firmware di test della rete	10
9.1	Inclusioni	10
9.2	Trasmissione sulla seriale USB	10
9.3	Contatore di cicli DWT	11
9.4	Ciclo di test	11
9.5	Caricamento e verifica	12

10 Prossimi passi	12
10.1 PWM per l'ESC	12
10.2 Restanti	12
11 Riepilogo delle lezioni	13

1 Obiettivo e architettura

Fino a questo punto la policy (rete *actor* di SAC, modello **F**) è stata eseguita sul PC: l'Arduino leggeva la cella di carico, mandava la misura al PC via USB, il PC calcolava l'azione e la rimandava all'Arduino, che pilotava l'ESC. L'obiettivo è eliminare il PC dall'anello:

timer 50 Hz -> leggi HX711 -> osservazione -> rete -> comando -> PWM all'ESC
(filtro EMA, integrale, ultimi 3 comandi)

Il periodo di controllo è $T = 20$ ms (50 Hz): tutto il ciclo deve stare in questo intervallo.

Effetto atteso sul ritardo. Con PC e Arduino il ritardo d'anello misurato sul banco corrispondeva a circa 3 passi (≈ 60 ms), dovuti a USB, scheduler di Windows e catena di comunicazione. Su un microcontrollore dedicato questi contributi spariscono; restano conversione dell'HX711, periodo del PWM e dinamica di ESC e motore. La policy F è stata addestrata con ritardo randomizzato tra 1 e 4 passi, quindi è adatta a entrambi i casi.

2 Scelta del microcontrollore

2.1 Dimensione della rete

L'actor è un percettrone a due strati nascosti con ReLU, $7 \rightarrow 256 \rightarrow 256 \rightarrow 1$. Il numero di parametri (pesi più bias) è

$$N_{par} = \underbrace{(7 \cdot 256 + 256)}_{\text{strato 1}} + \underbrace{(256 \cdot 256 + 256)}_{\text{strato 2}} + \underbrace{(256 + 1)}_{\text{uscita } \mu} = 2048 + 65\,792 + 257 = 68\,097. \quad (1)$$

In virgola mobile a singola precisione (4 byte):

$$M_{pesi} = 68\,097 \times 4 = 272\,388 \text{ B} \approx 266 \text{ KiB}. \quad (2)$$

Il numero di moltiplicazioni-somme (MAC) per un'inferenza è il numero dei pesi, esclusi i bias:

$$N_{MAC} = 7 \cdot 256 + 256 \cdot 256 + 256 = 67\,584. \quad (3)$$

2.2 Confronto tra le schede

	BluePill (STM32F103C8T6)	Black board (STM32F407VET6)
Core	Cortex-M3, 72 MHz	Cortex-M4F, 168 MHz
FPU	assente: float emulato in software	singola precisione in hardware
Flash	64 KB	512 KB
RAM	20 KB	128 KB + 64 KB CCM
La rete entra?	no (266 KiB > 64 KB)	sì, senza quantizzazione

Quali famiglie STM32 hanno la FPU La FPU c'è nei core Cortex-M4F/M7/M33 (famiglie F4, F7, G4, L4, H7...). Le famiglie F0, G0, L0 (Cortex-M0/M0+) e F1 (Cortex-M3) non ce l'hanno, anche quando hanno flash sufficiente: su di esse ogni operazione float costa decine di cicli. Una scheda NUCLEO-G0B1RE, per esempio, ha 512 KB di flash ma core M0+ a 64 MHz senza FPU: inadatta a questa rete.

2.3 Bilancio di tempo

Il tempo di un'inferenza dipende dai cicli spesi per ogni MAC (caricamento di peso e ingresso, moltiplicazione, somma) e dalle ottimizzazioni del compilatore. Anche in un caso pessimistico di 20 cicli per MAC:

$$t_{inf} \leq \frac{67584 \times 20}{168 \times 10^6 \text{ Hz}} \approx 8.0 \text{ ms} < T = 20 \text{ ms}. \quad (4)$$

Il valore reale viene misurato sul chip dal firmware di test (sezione 9).

3 La policy come funzione

3.1 Cosa calcola l'actor in modalità deterministica

In SAC l'actor produce una distribuzione gaussiana schiacciata da tanh: due teste lineari danno la media μ e il logaritmo della deviazione standard $\log \sigma$. In training l'azione si campiona; in esecuzione (`deterministic=True`) si usa direttamente la media:

$$a = \tanh\left(W_\mu \text{ReLU}(W_2 \text{ReLU}(W_1 x + b_1) + b_2) + b_\mu\right), \quad a \in [-1, 1]. \quad (5)$$

La testa $\log \sigma$ non serve e non viene esportata. Questo è stato verificato leggendo la struttura del modello con Stable-Baselines3:

```
Actor(
  (latent_pi): Sequential(
    (0): Linear(in_features=7, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=256, bias=True)
    (3): ReLU() )
  (mu): Linear(in_features=256, out_features=1, bias=True)
  (log_std): Linear(in_features=256, out_features=1, bias=True) )
```

3.2 Disposizione dei pesi in memoria

In PyTorch `Linear.weight` ha forma (`uscite × ingressi`) ed è memorizzato riga per riga (*row-major*). Uno strato calcola

$$y_r = b_r + \sum_{c=0}^{n_{in}-1} W_{rc} x_c, \quad r = 0, \dots, n_{out} - 1. \quad (6)$$

Appiattendo la matrice nello stesso ordine, l'elemento W_{rc} si trova all'indice

$$k = r \cdot n_{in} + c, \quad (7)$$

quindi la riga r è un blocco contiguo di n_{in} valori che inizia all'indice $r n_{in}$. Il codice C sfrutta proprio questo: punta all'inizio della riga e la scorre.

4 Esportazione della rete in C

4.1 Perché a mano e non con X-CUBE-AI

ST fornisce X-CUBE-AI, che genera il C da un modello ONNX. Per una rete di tre strati densi scrivere l'inferenza a mano è più semplice: sono tre prodotti matrice-vettore, nessuna dipendenza, e ogni riga è comprensibile e verificabile.

4.2 Script di esportazione

File `stm32/rete/esporta_actor.py`. Produce due header:

- `actor_weights.h`: tutti i pesi e i bias come array static `const float`;
- `test_vectors.h`: 100 osservazioni di prova e le azioni calcolate da PyTorch.

```

1 # =====
2 # esporta_actor.py -- trasforma l'actor SAC (Stable-Baselines3) in C per lo STM32
3 # =====
4 # Produce:
5 #   actor_weights.h : tutti i pesi e bias come array C costanti (finiscono in FLASH)
6 #   test_vectors.h : 100 osservazioni di test + l'azione calcolata da PyTorch,
7 #                   per verificare che il C dia lo stesso risultato
8 # Uso: python esporta_actor.py percorso\modello.zip
9 #      es. python esporta_actor.py ../../models\sac_F.zip
10 # =====
11 import sys
12 import numpy as np
13 import torch
14 from stable_baselines3 import SAC
15
16 model = SAC.load(sys.argv[1] if len(sys.argv) > 1 else "best_model.zip", device="cpu")
17 actor = model.actor
18
19 # --- 1. Estraggo i pesi. In inferenza deterministica servono solo latent_pi e mu: ---
20 #   azione = tanh( mu( relu(W2 relu(W1 x + b1) + b2) ) )
21 #   log_std serve solo per campionare in training: qui non lo esporto.
22 L1 = actor.latent_pi[0] # Linear 7 -> 256
23 L2 = actor.latent_pi[2] # Linear 256 -> 256
24 MU = actor.mu           # Linear 256 -> 1
25 strati = [("W1", L1.weight), ("B1", L1.bias), ("W2", L2.weight), ("B2", L2.bias),
26           ("WMU", MU.weight), ("BMU", MU.bias)]
27 n_in, n_h1, n_h2 = L1.in_features, L1.out_features, L2.out_features
28
29
30 def array_c(nome, t):
31     """Tensore -> array C 'static const float' piatto, in ordine riga per riga.
32     Per i pesi (out, in): l'elemento [r][c] sta in posizione r*in + c."""
33     v = t.detach().numpy().astype(np.float32).ravel()
34     righe = ["", ".join(f'{x:.9e}f' for x in v[i:i + 6]) for i in range(0, len(v), 6)]
35     return f"static const float {nome}[{len(v)}] = {{\n  " + ",\n  ".join(righe) + "\n};\n"
36
37
38 with open("actor_weights.h", "w") as f:
39     f.write("/* Generato da esporta_actor.py -- NON modificare a mano */\n")
40     f.write("#ifndef ACTOR_WEIGHTS_H\n#define ACTOR_WEIGHTS_H\n\n")
41     f.write(f"#define ACT_N_IN  {n_in}\n#define ACT_N_H1  {n_h1}\n#define ACT_N_H2  {n_h2}\n\n")
42     for nome, t in strati:
43         f.write(array_c(nome, t) + "\n")
44     f.write("#endif\n")
45
46 n_par = sum(t.numel() for _, t in strati)
47 print(f"actor_weights.h: {n_par} parametri = {n_par*4/1024:.1f} KB in flash")
48
49 # --- 2. Vettori di test: osservazioni realistiche nel range delle osservazioni ---
50 rng = np.random.default_rng(0)
51 lo, hi = model.observation_space.low, model.observation_space.high
52 obs = rng.uniform(np.maximum(lo, -1.5), np.minimum(hi, 1.5), size=(100, n_in)).astype(np.float32)
53
54 azioni, _ = model.predict(obs, deterministic=True) # stessa funzione usata sul banco
55 azioni = azioni.astype(np.float32).ravel()
56
57 with open("test_vectors.h", "w") as f:

```

```

57 f.write("/* Generato da esporta_actor.py */\n#ifndef TEST_VECTORS_H\n#define TEST_VECTORS_H\n\n")
58 f.write(f"#define N_TEST {len(obs)}\n\n")
59 f.write(array_c("TEST_OBS", torch.from_numpy(obs)) + "\n")
60 f.write(array_c("TEST_AZIONE", torch.from_numpy(azioni)) + "\n#endif\n")
61 print(f"test_vectors.h: {len(obs)} osservazioni, azioni in [{azioni.min():.3f}, {azioni.max():.3f}]\n")

```

Formato dei numeri. I pesi sono scritti con `%.9e`, cioè 10 cifre significative. Un float a 32 bit ha bisogno di 9 cifre decimali significative per essere riletto identico (`FLT_DECIMAL_DIG = 9`): con 10 il passaggio testo → binario è esatto.

Vettori di test. Le osservazioni sono estratte uniformemente nell'intervallo dello spazio delle osservazioni, limitato a $[-1.5, 1.5]$, con seme fisso (riproducibili). L'azione di riferimento è calcolata con `model.predict(obs, deterministic=True)`, la stessa funzione usata sul banco.

4.3 Inferenza in C

File `stm32/rete/actor.c`.

```

1  /* =====
2  * actor.c -- inferenza dell'actor SAC in C puro (nessuna libreria esterna)
3  * =====
4  * Rete: x(7) -> Linear 256 -> ReLU -> Linear 256 -> ReLU -> Linear 1 -> tanh
5  * Un "Linear" calcola y[r] = b[r] + somma_c W[r][c] * x[c] (prodotto matrice-vettore)
6  * I pesi sono in actor_weights.h come array piatti: W[r][c] = W[r*n_in + c].
7  * Operazioni: 7*256 + 256*256 + 256 = ~67.6k moltiplicazioni-somme (MAC).
8  * ===== */
9  #include <math.h>
10 #include "actor.h"
11 #include "actor_weights.h"
12
13 /* Buffer degli strati nascosti in RAM: 2 x 256 float = 2 KB.
14 * "static" li tiene fuori dallo stack (che sul micro e' piccolo). */
15 static float h1[ACT_N_H1];
16 static float h2[ACT_N_H2];
17
18 /* y = W x + b, con ReLU opzionale. W ha n_out righe e n_in colonne. */
19 static void lineare(const float *W, const float *b, const float *x, float *y,
20                   int n_in, int n_out, int relu)
21 {
22     for (int r = 0; r < n_out; r++) {
23         const float *riga = &W[r * n_in]; /* puntatore all'inizio della riga r */
24         float s = b[r]; /* parto dal bias */
25         for (int c = 0; c < n_in; c++) {
26             s += riga[c] * x[c]; /* una MAC: con la FPU e' 1 istruzione */
27         }
28         y[r] = (relu && s < 0.0f) ? 0.0f : s; /* ReLU: i negativi diventano 0 */
29     }
30 }
31
32 float actor_forward(const float *obs)
33 {
34     float mu;
35     lineare(W1, B1, obs, h1, ACT_N_IN, ACT_N_H1, 1); /* 7 -> 256, ReLU */
36     lineare(W2, B2, h1, h2, ACT_N_H1, ACT_N_H2, 1); /* 256 -> 256, ReLU */
37     lineare(WMU, BMU, h2, &mu, ACT_N_H2, 1, 0); /* 256 -> 1, lineare */
38     return tanhf(mu); /* schiaccia in [-1, 1] */
39 }

```

- `static const float W1[...]`: `const` dice al compilatore che i valori non cambiano, quindi il linker li mette nella sezione `.rodata`, che il linker script colloca in **flash**. Senza `const` i 266 KiB verrebbero copiati in RAM all'avvio, e non ci starebbero.
- `static float h1[256], h2[256]`: i vettori intermedi, $2 \times 256 \times 4 = 2$ KiB in RAM. Sono `static` (sezione `.bss`) e non variabili locali, perché lo stack riservato dal linker script è di soli $0x400 = 1024$ byte.
- `lineare()`: implementa $y = Wx + b$ con ReLU opzionale. L'unica funzione di libreria usata è `tanhf` (da `math.h`).

4.4 Verifica sul PC

File `stm32/rete/test_pc.c`: esegue `actor_forward` sui 100 vettori e confronta con PyTorch.

```

1  /* test_pc.c -- verifica sul PC: il C deve riprodurre PyTorch sui 100 vettori di test.
2  * Compila: gcc -O2 -o test_pc test_pc.c actor.c -lm */
3  #include <stdio.h>
4  #include <math.h>
5  #include "actor.h"
6  #include "actor_weights.h"
7  #include "test_vectors.h"
8
9  int main(void)
10 {
11     float err_max = 0.0f;
12     for (int k = 0; k < N_TEST; k++) {
13         float a = actor_forward(&TEST_OBS[k * ACT_N_IN]);
14         float e = fabsf(a - TEST_AZIONE[k]);
15         if (e > err_max) err_max = e;
16         if (k < 3) printf("test %d: C = %.6f   PyTorch = %.6f\n", k, a, TEST_AZIONE[k]);
17     }
18     printf("errore massimo su %d test: %.2e -> %s\n", N_TEST, err_max,
19           err_max < 1e-5f ? "OK" : "DIVERSO!");
20     return err_max < 1e-5f ? 0 : 1;
21 }
```

Risultato misurato (modello F)

```

actor_weights.h: 68097 parametri = 266.0 KB in flash
test_vectors.h: 100 osservazioni, azioni in [-1.000, 0.995]
test 0: C = -0.999328   PyTorch = -0.999328
test 1: C = +0.973925   PyTorch = +0.973925
test 2: C = +0.443066   PyTorch = +0.443066
errore massimo su 100 test: 3.87e-07 -> OK
```

L'esportazione è stata ripetuta sul PC di laboratorio a partire da `models/sac_F.zip` (impronta MD5 `F5BD5A72960522EAC260D9C42B191424`, identica al modello verificato) con lo stesso esito.

Perché l'errore non è zero, e perché $4 \cdot 10^{-7}$ è accettabile. In singola precisione l'unità di arrotondamento è $u = 2^{-24} \approx 5.96 \cdot 10^{-8}$. Un prodotto scalare di n termini sommati in ordine diverso può differire di una quantità dell'ordine di $nu \sum |w_c x_c|$. PyTorch e il C sommano in ordine diverso (PyTorch usa routine vettoriali), quindi piccole differenze sono inevitabili. L'errore osservato vale circa $6.5u$: è puro arrotondamento. La `tanh` ha derivata ≤ 1 , quindi non amplifica l'errore. Sul comando del motore, un'azione in $[-1, 1]$ diventa $u_{banco} \in [1, 20]$: $4 \cdot 10^{-7}$ in azione è circa $4 \cdot 10^{-6}$ in u , sei ordini di grandezza sotto la risoluzione dell'impulso (1 μ s su 928 μ s di corsa).

5 Clock a 168 MHz

La derivazione completa di ogni blocco della schermata *Clock Configuration* è nel documento dedicato docs/Clock_Configuration_STM32F407.pdf. Qui la sintesi.

5.1 Catena del PLL

Sorgente: quarzo esterno HSE da 8 MHz (obbligatorio con la USB: l'oscillatore interno HSI ha una tolleranza di circa $\pm 1\%$, la USB full-speed ne ammette $\pm 0.25\%$).

$$f_{in} = \frac{f_{HSE}}{M} = \frac{8}{8} = 1 \text{ MHz} \quad (1 \leq f_{in} \leq 2 \text{ MHz}) \quad (8)$$

$$f_{VCO} = f_{in} N = 336 \text{ MHz} \quad (100 \leq f_{VCO} \leq 432 \text{ MHz}) \quad (9)$$

$$f_{SYS} = \frac{f_{VCO}}{P} = \frac{336}{2} = 168 \text{ MHz} \quad (\leq 168 \text{ MHz}) \quad (10)$$

$$f_{USB} = \frac{f_{VCO}}{Q} = \frac{336}{7} = 48 \text{ MHz} \quad (= 48 \text{ MHz esatti}) \quad (11)$$

La scelta di f_{VCO} segue dal vincolo $f_{VCO} = 168 P = 48 Q$ con $P \in \{2, 4, 6, 8\}$: solo $P = 2$ dà $f_{VCO} = 336 \leq 432$, da cui $Q = 7$.

5.2 Bus

Bus	Prescaler	Frequenza	Massimo	Clock dei timer
AHB (CPU, memorie)	/1	168 MHz	168 MHz	—
APB1	/4	42 MHz	42 MHz	84 MHz
APB2	/2	84 MHz	84 MHz	168 MHz

Regola del raddoppio: se il prescaler di un bus APB è diverso da 1, i timer su quel bus ricevono il doppio della frequenza del bus.

5.3 Errore incontrato: $N = 136$

CubeMX mostrava tutto a 68 MHz e la casella dei 48 MHz in rosso con 19.43. Rifacendo la catena: $136/2 = 68$, $136/7 = 19.43$. Il valore sbagliato era N .

Wait state della flash A 168 MHz la flash richiede 5 cicli di attesa per lettura (con V_{DD} tra 2.7 e 3.6 V). CubeMX lo imposta da solo in `SystemClock_Config()` con `FLASH_LATENCY_5`; l'acceleratore ART (cache e prefetch) nasconde gran parte dell'attesa.

6 USB: caricamento (DFU) e comunicazione (CDC)

Scelta di progetto: un solo cavo, il connettore micro USB della scheda, senza ST-Link né adattatore USB-seriale. La stessa porta svolge due funzioni in momenti diversi, decisi dal pin **BOOT0**:

Momento	BOOT0	Chi gestisce la USB	Cosa vede il PC
Caricamento	3.3 V	bootloader di fabbrica (DFU)	dispositivo "STM32 BOOTLOADER"
Esecuzione	GND	il firmware (classe CDC)	una porta seriale COMx

Al reset il micro legge BOOT0: se è alto esegue il bootloader scritto in fabbrica nella *system memory*, se è basso salta al programma in flash (0x08000000). BOOT1 resta a GND.

La classe **CDC** (*Communication Device Class*, “Virtual Port Com”) fa apparire il micro come una porta seriale standard; Windows 10/11 ha già il driver. Il baud rate impostato sul PC è irrilevante, perché i dati viaggiano in pacchetti USB e non su una linea seriale reale.

Rinuncia: senza ST-Link non c’è debug passo-passo. Tutte le verifiche si fanno stampando sulla seriale.

Impostazioni CubeMX. USB_OTG_FS in modalità Device_Only (pin PA11 = D−, PA12 = D+); Middleware → USB_DEVICE con classe Communication Device Class; SYS → Debug: Serial Wire (PA13, PA14); PA6 come GPIO_Output (LED D2, attivo basso).

7 Il progetto CubeIDE

7.1 Struttura

File / cartella	Ruolo
FanStm32.ioc .project, .cproject	configurazione grafica di CubeMX (pin, clock, periferiche) descrizione del progetto per CubeIDE: compilatore, opzioni, percorsi degli header
Core/Src, Core/Inc Core/Startup/*.s	codice applicativo: main.c, actor.c e relativi header primo codice eseguito al reset: inizializza .data e .bss, chiama SystemInit e main
STM32F407VETX_FLASH.ld	linker script: mappa di flash e RAM e posizione di ogni sezione
Drivers/STM32F4xx_HAL_Driver	libreria HAL di ST
Drivers/CMSIS	definizioni del core e dei registri del chip (GPIOA, DWT, ...)
Middlewares, USB_DEVICE	stack USB di ST e sua configurazione per la classe CDC

7.2 Il linker script

Dal file STM32F407VETX_FLASH.ld:

```
CCMRAM (xrw) : ORIGIN = 0x10000000, LENGTH = 64K
RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 128K
FLASH (rx) : ORIGIN = 0x80000000, LENGTH = 512K
_Min_Heap_Size = 0x200; /* 512 B */
_Min_Stack_Size = 0x400; /* 1024 B */
```

I “192 KB di RAM” del datasheet sono quindi 128 KB di RAM principale più 64 KB di CCM RAM (*Core Coupled Memory*), collegata direttamente alla CPU e non accessibile dal DMA. Per default il linker usa solo la RAM principale.

7.3 Problemi incontrati nella creazione del progetto

1. **Toolchain sbagliata.** Il progetto era stato generato per IAR Embedded Workbench: nel file .ioc si leggeva TargetToolchain=EWARM V8.50. Mancavano .cproject, startup e linker script per GCC: CubeIDE lo vedeva come una cartella generica. *Soluzione:* Project Manager → Toolchain/IDE → STM32CubeIDE.
2. **.project preesistente.** La prima importazione aveva creato un .project generico; alla rigenerazione CubeMX lo ha trovato e non ha scritto gli altri file di CubeIDE (il log riportava

la generazione dei sorgenti ma nessun `.cproject`). *Soluzione*: rimozione del progetto dal workspace (senza cancellare i file), eliminazione di `.project`, nuova generazione.

3. **Residui della prima generazione.** In `Drivers/CMSIS` c'era l'intera libreria CMSIS (DSP, NN, RTOS, RTOS2, ...) e c'era la cartella `EWARM` con uno startup in sintassi IAR. CubeIDE compila tutti i `.c` presenti nel progetto: 15 errori, nessuno nel codice applicativo, del tipo

```
fatal error: RTE_Components.h: No such file or directory
fatal error: arm_nnfunctions.h: No such file or directory
```

Con `LibraryCopy=1` ("copia solo i file necessari") CubeMX copia solo `Device` e `Include`, ma non cancella ciò che resta da generazioni precedenti. *Soluzione*: cartelle spostate (non cancellate) in una cartella di backup fuori dal progetto.

Lezione Un errore di compilazione in un file che non si è mai toccato indica quasi sempre un problema di *progetto* (file in più, percorsi, toolchain), non di codice. Il primo controllo è il percorso del file che dà errore.

8 Compilazione

8.1 Dal sorgente all'eseguibile

1. **Compilazione:** ogni `.c` è tradotto separatamente in un file oggetto `.o` dal compilatore `arm-none-eabi-gcc`.
2. **Link:** il linker unisce gli oggetti, risolve i riferimenti tra file (`main.c` chiama `actor_forward` definita in `actor.c`) e assegna gli indirizzi seguendo il linker script. Risultato: `FanStm32.elf`.

Riga effettivamente usata per `actor.c` (configurazione Debug, percorsi di inclusione omessi):

```
arm-none-eabi-gcc ../Core/Src/actor.c -mcpu=cortex-m4 -std=gnu11 -g3 -DDEBUG
-DUSE_HAL_DRIVER -DSTM32F407xx -c -O0 -ffunction-sections -fdata-sections -Wall
--specs=nano.specs -mfpv4-sp-d16 -mfloat-abi=hard -mthumb
```

Opzione	Significato
<code>-mcpu=cortex-m4 -mthumb</code>	genera istruzioni per il Cortex-M4 (set Thumb-2)
<code>-mfpv4-sp-d16</code>	usa la FPU a singola precisione del M4
<code>-mfloat-abi=hard</code>	passa i float nei registri della FPU: i calcoli float sono istruzioni hardware
<code>-O0</code>	nessuna ottimizzazione (configurazione Debug): codice lento ma fedele al sorgente
<code>-ffunction-sections -fdata-sections</code>	ogni funzione e variabile in una sezione separata; con <code>-Wl,-gc-sections</code> il linker elimina quelle mai usate
<code>-specs=nano.specs</code>	libreria C ridotta (newlib-nano): il <code>printf</code> di default non stampa i float
<code>-g3</code>	informazioni di debug complete (non finiscono in flash)

8.2 Compilazione da riga di comando

CubeIDE ha una versione senza interfaccia, `stm32cubeidec.exe`, che compila un progetto su un workspace temporaneo; equivale al pulsante *Build*:

```
stm32cubeidec.exe --launcher.suppressErrors -nosplash
-application org.eclipse.cdt.managedbuilder.core.headlessbuild
-data <workspace temporaneo> -import <cartella del progetto> -build FanStm32/Debug
```

8.3 Occupazione di memoria

Uscita di arm-none-eabi-size

text	data	bss	dec	hex	filename
308156	332	11252	319740	4e0fc	FanStm32.elf

Build Finished. 0 errors, 0 warnings.

- **text**: codice e costanti, in flash.
- **data**: variabili con valore iniziale; occupano RAM, ma il loro valore iniziale è conservato in flash e copiato dallo startup.
- **bss**: variabili inizializzate a zero, solo in RAM.

$$\text{Flash} = \text{text} + \text{data} = 308\,488 \text{ B} \Rightarrow \frac{308\,488}{524\,288} = 58.8\% \quad (12)$$

$$\text{RAM} = \text{data} + \text{bss} = 11\,584 \text{ B} \Rightarrow \frac{11\,584}{131\,072} = 8.8\% \quad (13)$$

Ripartizione della flash: pesi 272 388 B, vettori di test $800 \times 4 = 3\,200$ B, il resto (≈ 32.6 KB) è codice: HAL, stack USB, libreria C, applicazione.

9 Firmware di test della rete

Scopo: ripetere sul microcontrollore la verifica fatta sul PC e misurare il tempo di inferenza. Tutto il codice applicativo è nei blocchi `/* USER CODE BEGIN ... */ ... /* USER CODE END ... */` di `main.c`: CubeMX, quando rigenera, conserva solo ciò che sta tra quei marcatori. È stato verificato: dopo due rigenerazioni il codice era intatto.

9.1 Inclusioni

```
1 /* USER CODE BEGIN Includes */
2 #include <stdio.h>
3 #include <string.h>
4 #include "usbd_cdc_if.h" // CDC_Transmit_FS: invio sulla seriale USB
5 #include "actor.h" // inferenza della rete (actor SAC)
6 #include "test_vectors.h" // 100 osservazioni + azioni calcolate da PyTorch
7 /* USER CODE END Includes */
```

9.2 Trasmissione sulla seriale USB

```
1 /* USER CODE BEGIN 0 */
2 #define N_OBS 7
3 extern USBDC_HandleTypeDef hUsbDeviceFS; // definito in usb_device.c
4 static char txbuf[160]; // buffer di uscita: deve restare valido
5 // finche' la USB non ha finito di spedirlo
6 static void usb_print(const char *s)
7 {
8     USBDC_HandleTypeDef *h = (USBDC_HandleTypeDef *)hUsbDeviceFS.pClassData;
9     uint32_t t0 = HAL_GetTick();
10     while (h != NULL && h->TxState != 0) { // invio precedente ancora in corso
11         if (HAL_GetTick() - t0 > 50) return; // PC non in ascolto: rinuncio
12     }
13 }
```

```

13  strncpy(txbuf, s, sizeof(txbuf) - 1);
14  CDC_Transmit_FS((uint8_t *)txbuf, strlen(txbuf));
15  }
16  /* USER CODE END 0 */

```

- CDC_Transmit_FS non copia i dati: avvia il trasferimento e ritorna subito; la periferica USB legge poi i byte direttamente dalla memoria. Il buffer deve quindi restare valido fino alla fine dell'invio: per questo è static. Una variabile locale verrebbe sovrascritta appena la funzione ritorna.
- TxState vale 0 quando la classe CDC è libera. Si aspetta al massimo 50 ms: se nessun programma sul PC ha aperto la porta, l'invio non si completa e il firmware non deve bloccarsi.
- hUsbDeviceFS.pClassData è NULL finché il PC non ha configurato il dispositivo: il controllo evita di dereferenziare un puntatore nullo.

9.3 Contatore di cicli DWT

```

1  /* USER CODE BEGIN 2 */
2  CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; // abilita le unita' di trace
3  DWT->CYCCNT = 0; // azzerare il contatore
4  DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk; // avvia il conteggio
5  HAL_Delay(2000); // tempo a Windows per riconoscere la porta seriale
6  /* USER CODE END 2 */

```

Il DWT (*Data Watchpoint and Trace*) è un'unità di debug del Cortex-M4 che contiene un contatore a 32 bit incrementato a ogni ciclo di clock. A 168 MHz:

$$1 \text{ ciclo} = \frac{1}{168} \mu\text{s} \approx 5.95 \text{ ns}, \quad t_{\text{overflow}} = \frac{2^{32}}{168 \times 10^6} \approx 25.6 \text{ s.} \quad (14)$$

La durata si calcola come CYCCNT_fine - CYCCNT_inizio in aritmetica uint32_t: la differenza tra interi senza segno è corretta anche se il contatore ha superato 2^{32} nel mezzo, purché l'intervallo misurato sia minore di 25.6 s.

9.4 Ciclo di test

```

1  /* USER CODE BEGIN 3 */
2  char riga[140];
3  float err_max = 0.0f;
4  uint32_t cicli = 0;
5  for (int k = 0; k < N_TEST; k++) {
6      uint32_t c0 = DWT->CYCCNT;
7      float a = actor_forward(&TEST_OBS[k * N_OBS]); // inferenza sul chip
8      cicli = DWT->CYCCNT - c0; // cicli dell'ultima inferenza
9      float e = a - TEST_AZIONE[k];
10     if (e < 0) e = -e;
11     if (e > err_max) err_max = e;
12 }
13 snprintf(riga, sizeof(riga),
14     "clock %lu MHz | errore max %lu e-9 | inferenza %lu cicli = %lu us | %s\r\n",
15     (unsigned long)(HAL_RCC_GetHCLKFreq() / 1000000), (unsigned long)(err_max * 1e9f),
16     (unsigned long)cicli, (unsigned long)(cicli / 168), err_max < 1e-5f ? "OK" : "
    DIVERSO!");
17 usb_print(riga);
18 HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6); // D2 lampeggia: il programma e' vivo
19 HAL_Delay(2000);
20 }
21 /* USER CODE END 3 */

```

- `TEST_OBS[k * N_OBS]`: le osservazioni sono un array piatto di 100×7 valori; la k -esima inizia all'indice $7k$.
- `HAL_RCC_GetHCLKFreq()` legge dai registri RCC la frequenza effettiva: è la prova che il clock è davvero 168 MHz, non solo nel disegno di CubeMX.
- L'errore è stampato come intero $\times 10^9$ perché con `nano.specs` il formato `%f` non è abilitato (si può attivare con l'opzione di link `-u _printf_float`, al prezzo di qualche KB di flash).
- Criterio di accettazione: errore massimo $< 10^{-5}$, lo stesso usato sul PC.

9.5 Caricamento e verifica

1. BOOT0 su 3.3 V, collegare la USB.
2. STM32CubeProgrammer: porta USB, *Connect*; poi *Erasing & Programming*, scegliere il file `Debug/FanStm32.elf` e premere *Start Programming*.
3. BOOT0 su GND, scollegare e ricollegare.
4. Aprire la nuova porta COMx con un terminale seriale (es. Monitor seriale dell'IDE Arduino).

Riga attesa ogni 2 s, con il LED D2 che cambia stato:

```
clock 168 MHz | errore max ... e-9 | inferenza ... cicli = ... us | OK
```

Risultato sul chip Da completare dopo la prova sulla scheda: frequenza letta, errore massimo, cicli e microsecondi per inferenza. Atteso: `clock 168`, errore dell'ordine di 10^{-7} come sul PC, tempo di inferenza nell'ordine dei millisecondi con `-00`.

10 Prossimi passi

10.1 PWM per l'ESC

L'ESC accetta un impulso ogni 20 ms (50 Hz). Il comando usato sul banco va da 1472 μ s (fermo) a 544 μ s (pieno). Con un timer su APB1 ($f_{TIM} = 84$ MHz):

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1)(ARR + 1)}. \quad (15)$$

Con $PSC = 83$ il contatore avanza a $84/84 = 1$ MHz, cioè 1 μ s per conteggio; con $ARR = 19999$ il periodo è 20000 μ s = 20 ms. Il registro di confronto CCR è allora direttamente la larghezza dell'impulso in microsecondi:

$$CCR = 1472 - \frac{u}{40} (1472 - 544), \quad u \in [0, 40]. \quad (16)$$

10.2 Restanti

1. Lettura dell'HX711 (stesso algoritmo del firmware Arduino v2, senza bloccare gli interrupt) e nuova calibrazione se l'HX711 viene alimentato a 3.3 V invece che a 5 V.
2. Anello di controllo in un interrupt a 50 Hz: osservazione (forza, errore, derivata filtrata, ultimi 3 comandi, integrale) costruita esattamente come in `env_fan.py`, inferenza, comando.
3. Protezioni: campione non valido, sensore muto per più di 100 ms, forza oltre 8 N, limite sul comando.
4. Prova sul banco con lo stesso protocollo usato per PC + Arduino e confronto dei risultati.

11 Riepilogo delle lezioni

- Verificare la rete esportata contro il modello originale *prima* di metterla sul chip: separa gli errori di traduzione da quelli di firmware.
- Nei microcontrollori le costanti grandi vanno dichiarate `const`, altrimenti finiscono in RAM.
- Il clock si progetta da sinistra a destra rispettando prima le frequenze obbligate (USB 48 MHz), poi quella desiderata; si verifica sul chip leggendo i registri.
- In CubeMX la toolchain di destinazione determina quali file di progetto vengono creati; file residui di generazioni precedenti non vengono rimossi.
- Il codice applicativo va sempre nei blocchi `USER CODE`.